

Active Fault-Tolerant Control Based on MPC and Reinforcement Learning for Quadcopter with Actuator Faults^{*}

Huicheng Jiang^{*} Feng Xu^{*,****} Xueqian Wang^{*}
Songtao Wang^{**,***}

^{*} *Tsinghua Shenzhen International Graduate, Tsinghua University,
518055 Shenzhen, P.R.China*

^{**} *School of Mechanical Engineering, Nanchang Institute of
Technology, 330099 Nanchang, P.R.China*

^{***} *Research Institute of Tsinghua University in Shenzhen, 518057
Shenzhen, P.R.China*

^{****} *Corresponding author: xu.feng@sz.tsinghua.edu.cn*

Abstract: In this paper, we propose an active fault-tolerant control (AFTC) method combining model predictive control (MPC) and reinforcement learning (RL) for the quadcopter with actuator faults. We take a data-based discriminant model as the fault detection and diagnosis (FDD) module indicating a system fault mode based on the state error. With the information of the fault mode and the state error, the RL controller generates auxiliary control signals to correct

Copyright © 2023 The Authors. This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>)

Keywords: Learning for control, quadcopter, actuator faults, active fault-tolerant control, model predictive control, reinforcement learning

1. INTRODUCTION

Due to their flexible and autonomous flight capabilities, quadcopters are widely used in many scenarios, such as military reconnaissance, pesticide spraying, filming, etc. With the increasing interaction between quadcopters and human, more attention has been paid to their safety and reliability, that is, the ability to maintain normal operation in the event of disturbances and faults (Fourlas and Karras (2021)). In this background, many fault-tolerant control (FTC) methods have been applied to quadcopters. In general, these existing FTC methods can be divided into three categories: model-based, learning-based, and hybrid control.

As a classical model-based control method, MPC is often used to construct fault-tolerant controllers. The work (El-trabyly et al. (2021b)) proposed an MPC controller based on the quadcopter's dynamic model, and reconfigured it with actuator fault parameters estimated by nonlinear moving horizon estimation (NMHE) to achieve its fault tolerance. Similar to this thread, the same result can be achieved by combining various parameter estimation

methods or observations with the MPC controller (El-trabyly et al. (2021a), Yu et al. (2015)). With an MPC controller as the main controller, the author (Nan et al. (2022)) constructed an auxiliary controller according to the quadcopter's nonlinear incremental dynamics model generating the compensation torque to achieve FTC under actuator faults. However, the fault-tolerant performance of model-based FTC methods is highly dependent on accurate modeling and takes a lot of real-time computing power to estimate the parameters.

Unlike the model-based approach, as a representative of unsupervised learning, the RL controller learns the optimal control strategy by interacting with the environment, without relying on accurate modeling. At present, RL has been applied to the quadcopter for path following (Rubí et al. (2020), Hwangbo et al. (2017)). An FTC method for the quadcopter based on RL and directly generating compensation torque through state error was proposed (Fei et al. (2020)). However, the system instability will increase when this compensation input continues to act on the quadcopter.

Compared with the first two categories, the idea of hybrid control can reduce the requirements of accurate modeling, and speed up real-time response while improve the stability that cannot be guaranteed by learning-based methods. This FTC method makes it possible to apply the learning-

^{*} This work was supported by the National Natural Science Foundation of China (No.62003186, No. 52265002), the Natural Science Foundation of Guangdong, China (No. 2021A1515012628), and the Shenzhen Science and Technology Program, China (No.JCYJ20210324132606015).

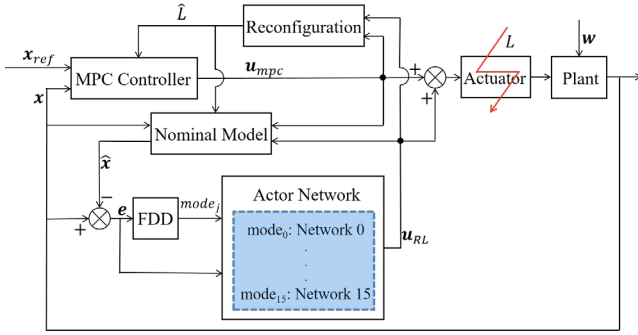


Fig. 1. The structure of AFTC

based algorithm to practice. A hybrid FTC framework which combined PID controller with RL controller was proposed (Sohège et al. (2021)). The RL controller as a high-level controller was trained under different operating conditions to adjust the parameters of the PID controller which was responsible for the trajectory tracking of the quadcopter. However, there are several problems with this FTC approach. First, due to the lack of fault detection and diagnosis (FDD) modules, there is a lack of connection between PID parameters and fault parameters. Second, the RL controller directly affects the parameters of the PID controller through the state error and is always on, so the system stability cannot be completely guaranteed.

In order to integrate the stability advantage of model-based FTC with the low model dependency and real-time computing advantages of learning-based FTC, we propose a novel hybrid AFTC framework combining MPC with RL for the quadcopter with actuator faults in this paper (Fig.1). As the core of this framework, we propose an auxiliary signal-based actuator fault parameter estimation method and prove its convergence. When faults occur, the RL controller generates an auxiliary control signal according to the fault mode and the state error to maintain the attitude stabilization and update the estimation of fault parameters. The MPC controller is configured with the parameter estimation. And then the RL controller gradually quits control of the system with the gradual matching between the controller model and the real model.

The paper is structured as follows. The dynamic model of the quadcopter with fault representation is briefly introduced in Section 2. In Section 3, we introduce the modeling process of the MPC controller, the FDD module and the auxiliary controller, and propose the fault parameters estimation method. In Section 4, we show the training results of the FDD module and the auxiliary controller, and the results of two trajectory tracking simulations under AFTC. Finally, a conclusion is given in Section 5.

2. QUADCOPTER MODEL WITH FAULT REPRESENTATION

The structure of the quadcopter is shown in Fig.2, including four motors and propellers. In this section, the dynamic model (Luukkonen (2011)) and actuator failure model of the quadcopter are briefly introduced.

2.1 Notations

- Vectors are depicted by bolded letters.

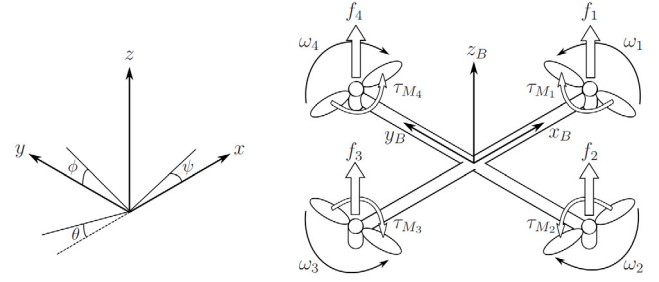


Fig. 2. Rotors' numbers, directions of rotation, reference frames and euler angles

- Matrices are designated by upper-case letters.
- Scalars are denoted by non-bolded, lower-case letters.
- Superscripts B and I represent the body and inertial frames, respectively.

2.2 Kinematics and Dynamics Models

The quadcopter's model can be derived from Newton Euler equation as follows:

$$\begin{cases} \dot{\zeta} = R_{IB}V, & (1a) \\ \dot{u} = gS_\theta + qw - rv - k_x u/m, & (1b) \\ \dot{v} = -gS_\phi C_\theta + ru - pw - k_y v/m, & (1c) \\ \dot{w} = F_{th,z}^B/m + pv - qu - gC_\phi C_\theta - k_z w/m, & (1d) \\ \dot{\eta} = T_{IB}\omega, & (1e) \\ \dot{p}I_{xx} = \tau_{th,x}^B + (I_{yy} - I_{zz})qr - k_p p, & (1f) \\ \dot{q}I_{yy} = \tau_{th,y}^B + (I_{zz} - I_{xx})pr - k_q q, & (1g) \\ \dot{r}I_{zz} = \tau_{th,z}^B + (I_{xx} - I_{yy})pq - k_r r. & (1h) \end{cases}$$

The position of the quadcopter is defined in the inertial frame with $\zeta = [x \ y \ z]^T$, and the attitude is described as $\eta = [\phi \ \theta \ \psi]^T$ using Euler angles (roll, pitch, and yaw angles). $V = [u \ v \ w]^T$ and $\omega = [p \ q \ r]^T$ represent the linear velocities and the angular velocities in the body frame, respectively. g is the gravitational acceleration and m is the mass of quadcopter. k_x, k_y, k_z are translational drag coefficients and k_p, k_q, k_r are rotational drag coefficients, respectively. k_{xx}, k_{yy}, k_{zz} are the moments of inertia about the x, y and z axes, respectively. $F_{th,z}^B$ is thrust force along the z-axes in the body frame, and $\tau_{th,x}^B, \tau_{th,y}^B$ and $\tau_{th,z}^B$ are the moments about the x, y and z axes, respectively. The details of $F_{th,z}^B, \tau_{th,x}^B, \tau_{th,y}^B$ and $\tau_{th,z}^B$ are further introduced in the equation(2b). R_{IB} and T_{IB} are rotation and transformation matrices (Luukkonen (2011)), respectively. S_θ, C_θ and T_θ are abbreviations of $\sin(\theta), \cos(\theta)$ and $\tan(\theta)$, respectively, and the same for ϕ and ψ .

2.3 Multiplicative Fault Representation

In order to obtain the actuator fault model, we modify the equations (1d) and (1f)-(1h) and introduce the fault matrix L :

$$\begin{bmatrix} \dot{w} \\ \dot{p}I_{xx} \\ \dot{q}I_{yy} \\ \dot{r}I_{zz} \end{bmatrix} = MLu + \begin{bmatrix} m(pv - qu) - mgC_\phi C_\theta - k_z w \\ (I_{yy} - I_{zz})qr - k_p p \\ (I_{zz} - I_{xx})pr - k_q q \\ (I_{xx} - I_{yy})pq - k_r r \end{bmatrix}, \quad (2a)$$

$$\begin{bmatrix} F_{th,z}^B \\ \tau_{th,x}^B \\ \tau_{th,y}^B \\ \tau_{th,z}^B \end{bmatrix} = \underbrace{\begin{bmatrix} k_T & k_T & k_T & k_T \\ 0 & -k_T l & 0 & k_T l \\ -k_T l & 0 & k_T l & 0 \\ -b & b & -b & b \end{bmatrix}}_M \underbrace{L}_{\mathbf{u}} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}. \quad (2b)$$

Where $\mathbf{u} \in \mathbb{R}^{n_u}$ is the control action vector in (rpm)² with n_u being the number of control inputs, and M is the allocation matrix, mapping $\mathbf{u}$ to the thrust force and torques in the body frame. k_T is the thrust constant and b is the drag constant. Matrix $L = \text{diag}(l_1, l_2, l_3, l_4)$ denotes the efficiency of actuators (i.e., modeling the effect of actuator faults). The efficiency of the i th actuator varies in the range $0 \leq l_i \leq 1$, ($i = 1, \dots, 4$), where $l_i = 0$ depicts a complete actuator failure, $l_i = 1$ a healthy actuator, and $0 < l_i < 1$ a partial loss of actuator efficiency. From (1), (2a) and (2b), the nonlinear model of the quadcopter with faults, disturbances and measurement noise is described as follows:

$$\begin{cases} \dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}, L) + \mathbf{w}, \\ \mathbf{y} = C\mathbf{x} + \mathbf{v} \end{cases} \quad (3a)$$

$$(3b)$$

with $\mathbf{x} = [\zeta^T, \mathbf{V}^T, \boldsymbol{\eta}^T, \boldsymbol{\omega}^T]^T \in \mathbb{R}^{n_x}$ the state vector, and n_x the number of states. $\mathbf{y} \in \mathbb{R}^{n_y}$, with n_y the number of outputs. $\mathbf{w} \in \mathbb{R}^{n_x}$ is the system disturbance and $\mathbf{v} \in \mathbb{R}^{n_y}$ is the random measurement noise. f is a nonlinear function.

3. FDD AND FTC FOR QUADCOPTER WITH ACTUATOR FAULTS

3.1 Trajectory Tracking with MPC

The MPC controller acts as the main controller for trajectory tracking by solving the optimization problem, and the nonlinear Optimal Control Problem (OCP) is formulated as

$$\begin{aligned} \min_{\mathbf{U}, \mathbf{X}} J(\mathbf{x}_k, \mathbf{U}, \mathbf{X}^{\text{ref}}) = \\ \sum_{i=0}^{N_p-1} \left(\|\tilde{\mathbf{x}}_{k+i|k}\|_Q^2 + \|\mathbf{u}_{k+i|k}\|_R^2 + \|\Delta \mathbf{u}_{k+i|k}\|_{R_\Delta}^2 \right) \\ + \|\tilde{\mathbf{x}}_{k+N_p|k}\|_{Q_N}^2 \end{aligned} \quad (4)$$

s.t.

$$\mathbf{x}_{k+i+1|k} = f_d(\mathbf{x}_{k+i|k}, \mathbf{u}_{k+i|k}, \hat{L}_k), \quad (5a)$$

$$\tilde{\mathbf{x}}_{k+i|k} = \mathbf{x}_{k+i|k} - \mathbf{x}_{k+i}^{\text{ref}}, \quad (5b)$$

$$\Delta \mathbf{u}_{k+i|k} = \mathbf{u}_{k+i|k} - \mathbf{u}_{k+i-1|k}, \quad (5c)$$

$$\mathbf{x}_{\min} \leq \mathbf{x}_{k+i|k} \leq \mathbf{x}_{\max}, \quad (5d)$$

$$\mathbf{u}_{\min} \leq \mathbf{u}_{k+i|k} \leq \mathbf{u}_{\max}, \quad (5e)$$

$$\mathbf{x}_{k|k} = \mathbf{x}_k, \quad (5f)$$

for $i = 0, 1, \dots, N_p - 1$, and N_p is the prediction horizon. f_d is the discrete form of the dynamic model f in (3a). $\hat{L} = \text{diag}(\hat{l}_1, \hat{l}_2, \hat{l}_3, \hat{l}_4)$ is the estimation of actuators effectiveness introduced later. $\mathbf{x}_{k+i|k}$ is the prediction of $\mathbf{x}(k+i)$ calculated based on the measurement of state at time instant k , and similarly for $\mathbf{u}_{k+i|k}$. $\mathbf{x}_{k|k}$ is the initial value of the optimization problem at time instant k . $Q, Q_N \in \mathbb{R}^{n_x \times n_x}$, R and $R_\Delta \in \mathbb{R}^{n_u \times n_u}$ are diagonal positive definite weighting matrices. $\mathbf{U} = \{\mathbf{u}_{k+i|k} \mid i = 0, 1, \dots, N_p - 1\}$

is the control input sequence and only $\mathbf{u}_{k|k}$ will act on system, $\mathbf{X} = \{\mathbf{x}_{k+i|k} \mid i = 0, 1, \dots, N_p\}$ is predicted state sequence, and $\mathbf{X}^{\text{ref}} = \{\mathbf{x}_{k+i}^{\text{ref}} \mid i = 0, 1, \dots, N_p\}$ is the reference trajectory.

In the optimization problem, we take the state error $\tilde{\mathbf{x}}$ as the cost to guarantee the stability of MPC, and the change $\Delta \mathbf{u}$ of input as the cost to smooth the input signal.

3.2 FDD Scheme Based on Classification Algorithm

In this subsection, we will introduce an FDD method for quadcopter with actuator faults based on classification of the state error. In different cases of actuator faults, the quadcopter takes on different characteristics and we can get fault information from its performance (i.e., state error). On one hand, the state estimation methods for the quadcopter have been extensively studied. On the other hand, due to the necessity of the state information for the MPC controller (4), we assume that the states of the quadcopter are measurable in this paper.

Sampling: To obtain the data set of the state error, we conduct random sampling based on models of different fault modes:

$$\mathbf{x}_{h,s}^+ = f_d(\mathbf{x}_s, \mathbf{u}_s, L = I_{n_u \times n_u}), \quad (6a)$$

$$\mathbf{x}_{f_{j,q},s}^+ = f_d(\mathbf{x}_s, \mathbf{u}_s, L_{\text{mode}_{j,q}}), \quad (6b)$$

$$\mathbf{e}_{s,j,q} = R_{IB}^{-1}(\mathbf{x}_{f_{j,q},s}^+ - \mathbf{x}_{h,s}^+), \quad (6c)$$

where

$$L_{\text{mode}_{j,q}} = \text{diag}(l_{1,j,q}, l_{2,j,q}, l_{3,j,q}, l_{4,j,q}), \quad (6d)$$

$$l_{m,j,q} \sim \begin{cases} \mathcal{U}(0.2, 0.98), & \text{motor } m \text{ is faulty,} \\ \mathcal{U}(0.98, 1), & \text{motor } m \text{ is healthy.} \end{cases} \quad (6e)$$

$$\mathbf{x}_s \in \mathbb{X}, \mathbf{x}_s \sim \mathcal{N}(0, \Sigma), \quad (6f)$$

$$\mathbf{u}_s \in \mathbb{U}, s = 1, 2, \dots, N_s,$$

$$j = 0, 1, \dots, 2^4 - 1, \text{ and } q = 1, 2, \dots, N_q.$$

In (6), $\mathbf{x}_s$ is the random state sampled from the state space $\mathbb{X}$ according to Gaussian distribution $\mathcal{N}(0, \Sigma)$ where Σ is a diagonal matrix. It is worth noting that, in order to make the sampling results general, a normal distribution with zero mean is used here to represent the state space of the quadcopter (the quadcopter often works in a state of small attitude angles and small angular velocities, and the translation for its position has no effect on sampling for the state error). According to whether a motor is faulty, the modes of the quadcopter can be divided into 2^4 types¹ (one healthy mode and 15 fault modes) represent by mode_j . $\text{mode}_{j,q}$ represents a sub-mode of mode_j , $L_{\text{mode}_{j,q}}$ is a matrix of random fault parameters sampled from the uniform distribution² $\mathcal{U}$, and N_q is the number of sub-modes in each fault mode. N_s represents the number of samples in each sub-mode. $\mathbf{x}_{h,s}^+$ and $\mathbf{x}_{f_{j,q},s}^+$

¹ In this paper, the binary number corresponding to the subscript j is used to indicate whether the motor is faulty, 0 indicates failure, and 1 indicates health, e.g. mode_0 represents that all motors are faulty and mode_7 means that motor 1 is faulty.

² Considering that there is almost no case where l_m is exactly equal to 1 in practice, the interval of l_m for a healthy motor is set to $[0.98, 1]$. When l_m is close to 0, the motor is close to complete failure and an extreme FTC strategy should be taken, and this case does not belong to the research content of this paper. Therefore, the interval of l_m for a faulty motor is set to $[0.2, 0.98]$.

are the successor states of a random sampled state x_s for the normal and fault modes with the same input u_s , respectively. u_s is an appropriate input for state x_s which can be generated by the controller (4) for convenience:

$$u_s = \arg \min J(x_s, U, 0) |_{\hat{L}=I_{n_u \times n_u}}, \quad (7)$$

where 0 is the zero vector sequence. The state error can be obtained by (6c) and we can get the data sets of state errors as follows:

$$\mathcal{S}_j = \{e_{i,j,q} | i = 1, 2, \dots, N_i, q = 1, 2, \dots, N_q\}, \quad (8a)$$

$$\mathcal{S} = \{\mathcal{S}_j | j = 0, 1, \dots, 2^4 - 1\}, \quad (8b)$$

where $\mathcal{S}_j$ is the state error set of $mode_j$, and $\mathcal{S}$ is the set of $\mathcal{S}_j$ and contains the total samples.

Classification and Discrimination: Taking $\mathcal{S}$ as the training data and $mode_j$ as the label, the discrimination model can be obtained through classification algorithm. Before training, standardization and Principal Component Analysis (PCA) algorithms are used to reduce the dimensions of the dataset. The dataset owns multiple feature dimensions and the distribution of samples in each mode is nonlinear. Therefore, we use Support Vector Classification (SVC) algorithm (Gunn et al. (1998)) for classification. When the quadcopter is running online, we judge the current fault mode using the discrimination model and real-time state error after projection transformation.

3.3 Auxiliary Controller Based on RL

In this subsection, an auxiliary controller based on deep deterministic policy gradient (DDPG) algorithm is introduced. With the information of the fault mode and the state error, the auxiliary controller generates the corresponding auxiliary control signal which acts on quadcopter system. The control system structure is shown in Fig.1.

Reinforcement Learning: In reinforcement learning, the agent learns the optimal policy π^* in interaction with the environment. At discrete time instant k , the agent receives a state s_k and chooses the action $a_k = \pi_\theta(s_k)$ via the parameterized policy π_θ . Then the agent gets a reward r_k and the next state s_{k+1} from environment E . The cumulative discounted reward is defined as $R_k = \sum_{i=k}^T \gamma^{i-k} r_i$ and the expectation of rewards $J = E_{s \sim E, a \sim \pi} [R_0]$ is taken as the optimization objective.

Problem Setup: The optimization of auxiliary controller can be formulated with a standard RL problem. We construct a simulation environment and take the auxiliary controller as the agent. The state and the action of the agent under $mode_j$ are denoted as (9c) and (9d):

$$\hat{x}_k = f_d(x_{k-1}, u_{mpc,k-1} + u_{RL,k-1}, \hat{L}_k), \quad (9a)$$

$$e_k = R_{IB}^{-1}(x_k - \hat{x}_k), \quad (9b)$$

$$s_k = e_k, \quad s_k \in \mathbb{S}, \quad \mathbb{S} \subset \mathbb{R}^{n_u}, \quad (9c)$$

$$a_k = [a_{1,k}, \dots, a_{n_u^j,k}], \quad a_k \in \mathbb{A}_j, \quad \mathbb{A}_j \subset \mathbb{R}^{n_u^j}, \quad (9d)$$

where n_u^j is the number of faulty motors in $mode_j$ ($j = 0, 1, \dots, 2^4 - 1$), $\hat{x}_k$ is the estimation of quadcopter's state at time k by the nominal model and x_k is the measurement state. The state of the agent s_k is denoted by the error between $\hat{x}_k$ and x_k , and $\mathbb{S}$ is the state space of the agent. a_k is the action for n_u^j faulty motors and $\mathbb{A}_j$ is the

action space under $mode_j$. In each episode q under $mode_j$, the fault parameter matrix $L_{mode_j,q}$ of the quadcopter is initialized by (6d) and (6e). $\hat{x}_k$ and x_k are initialized by (6f) and $\hat{x}_0 = x_0$. The agent takes an action a_k as

$$a_k \sim \pi_\theta(a_k | e_k), \quad (10)$$

where π_θ is a deterministic policy. In order to improve agent's exploration of action space, we can introduce a Gaussian noise $\epsilon_k \sim \mathcal{N}(0, \Sigma_{n_u^j \times n_u^j})$ to a_k . The auxiliary control signal $u_{RL,k} \in \mathbb{R}^4$ can be constructed based on a_k :

$$u_{RL,k} = [\tilde{u}_{1,k}, \dots, \tilde{u}_{4,k}], \quad (11a)$$

where $\tilde{u}_{m,k} = \begin{cases} 0, & \text{if motor } m \text{ is healthy in } mode_j, \\ a, \quad a \in a_k, & \text{if motor } m \text{ is faulty in } mode_j, \end{cases}$

and $m = 1, \dots, 4$.

The MPC controller is needed as the main controller during training of the agent, and the main control signal is obtained by

$$u_{mpc,k} = \arg \min J(x_k, U, 0) |_{\hat{L}=\hat{L}_k}. \quad (12)$$

The total input acts on the quadcopter is

$$u_k = L(u_{mpc,k} + u_{RL,k}). \quad (13)$$

We can get the next state x_{k+1} of the quadcopter with the control signal u_k , the next estimation $\hat{x}_{k+1}$ by (9a) and the next state of agent s_{k+1} by (9b) and (9c).

The objective of the auxiliary controller is to reduce the state error of the system, so the reward for each step can be constructed as follows:

$$r_k = \|s_k\|_\Lambda^2 - \|s_{k+1}\|_\Lambda^2, \quad (14)$$

where Λ is a weight matrix. During each episode, the total reward is the sum of the discounted reward $R = \sum_{k=0}^T \gamma^k r_k$ and T is the maximum steps for each episode. Taking $\gamma = 1$, it can be found that R takes the maximum and is exactly 0 when the error is fully compensated. The goal of the RL task is to find optimal policy π_θ , equal to maximize the average total reward, and can be written as follows:

$$\pi_\theta^* = \arg \max_\theta E_{a_k \sim \pi_\theta} [R]. \quad (15)$$

Training: The tuple used for training can be denoted as (s_k, a_k, r_k, s_{k+1}) by (9c), (9d) and (14). Since the system dynamic model is deterministic, we adopt the DDPG algorithm (Lillicrap et al. (2015)) which owns an Actor-Critic structure for training the agent (auxiliary controller). And the specific details and results will be given in the following contents.

3.4 AFTC Combined with MPC and RL Controllers

If the auxiliary signal u_{RL} directly acts to the system continuously, the system may run out of control due to the instability of the RL controller. In case of a large fault in the system, the auxiliary signal will increase, which will enhance the trend of system instability. Therefore, an AFTC framework shown in Fig.1 is needed which accommodates the MPC controller and nominal model to real system, reducing the state error which acts as the input of the RL controller, and weakening the impact of instability caused by the RL controller.

The key of AFTC is to reconfigure the controller according to the estimation of fault parameters. To do this in real

time, we obtain the estimation of the fault parameter $\hat{L}$ based on the control signals $\mathbf{u}_{mpc}$ and $\mathbf{u}_{RL}$ not depending on the quadcopter model as

$$\begin{aligned}\hat{l}_{m,k} &= \Pi_{i=0}^{k-1} \frac{(\mathbf{u}_{mpc,\hat{L}_i})_m}{(\mathbf{u}_{mpc,\hat{L}_i} + \mathbf{u}_{RL,i})_m} \\ &= \hat{l}_{m,k-1} \frac{(\mathbf{u}_{mpc,\hat{L}_{k-1}})_m}{(\mathbf{u}_{mpc,\hat{L}_{k-1}} + \mathbf{u}_{RL,k-1})_m}, \quad m = 1, \dots, 4,\end{aligned}\quad (16)$$

where the subscript m is the motor label, $(\cdot)_m$ represents the m th element of the vector, and $\mathbf{u}_{mpc,\hat{L}_i}$ is the control signal from MPC with $\hat{L} = \hat{L}_i$ at time instant i . Then we can reconfigure the MPC controller and the nominal model with $\hat{L}_k$.

Proposition 1. With the negative feedback mechanism in the AFTC framework and the appropriate auxiliary control signal $\mathbf{u}_{RL}$ reducing the state error, the difference between the fault parameter estimation by (16) and the real fault parameter will converge into a bounded range.

Proof. At time instant k , $\hat{L} = \hat{L}_k$ and the real input $\mathbf{u}_k$ and the desired input $\mathbf{u}_{d,k}$ of the system are

$$\mathbf{u}_k = L(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k}) + \boldsymbol{\omega}_{u,k}, \quad (17a)$$

$$\mathbf{u}_{d,k} = L(\mathbf{u}_{mpc,\hat{L}_k} + \tilde{\mathbf{u}}_{d,k}), \quad (17b)$$

where $\boldsymbol{\omega}_{u,k}$ is the actuator noise and $\tilde{\mathbf{u}}_{d,k}$ is the desired auxiliary input. Because the auxiliary input can not fully compensate the state error caused by faults, there are differences between $\mathbf{u}_{RL,k}$ and the desired auxiliary input $\tilde{\mathbf{u}}_{d,k}$:

$$\mathbf{e}_{\tilde{u},k} = \tilde{\mathbf{u}}_{d,k} - \mathbf{u}_{RL,k}. \quad (18)$$

So the desired input can be represented by $\mathbf{u}_{mpc,\hat{L}_k}$ and $\mathbf{u}_{RL,k}$ as (19):

$$\mathbf{u}_{d,k} = L(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k} + \mathbf{e}_{\tilde{u},k}). \quad (19)$$

Considering the case that the estimation of fault parameters is consistent with the real fault in the system ($\hat{L} = L$), the state error is completely determined by system disturbances and noise. So the desired auxiliary input can be viewed as a variable related to the disturbance, $\tilde{\mathbf{u}}_{d,k} = \tilde{\mathbf{u}}_{\omega_k}$ and the desired input can be written as

$$\mathbf{u}_{d,k} = L(\mathbf{u}_{mpc,L} + \tilde{\mathbf{u}}_{\omega_k}), \quad (20)$$

where $\tilde{\mathbf{u}}_{\omega_k}$ is the bounded auxiliary input for bounded system disturbances.

Considering the general case $\hat{L}_k = LP_k^{-1}$ that the matrix P_k is a positive definite diagonal matrix, the following propositions (21a) and (21b) from (4) and (5a) are established under the actuator fault model (2a):

$$f_d(\mathbf{x}_k, \mathbf{u}_k, L) \iff f_d(\mathbf{x}_k, P_k \mathbf{u}_k, \hat{L}_k), \quad (21a)$$

$$\begin{aligned}& \min_{U, \mathbf{X}} J(\mathbf{x}_k, U, \mathbf{X}^{\text{ref}}) |_L \\ & \iff \min_{U, \mathbf{X}} J(\mathbf{x}_k, P_k U, \mathbf{X}^{\text{ref}}) |_{\hat{L}_k}.\end{aligned}\quad (21b)$$

Based on the above propositions, we can establish a relationship between $\mathbf{u}_{mpc,\hat{L}_k}$ and $\mathbf{u}_{mpc,L}$ in (22):

$$\begin{aligned}\mathbf{u}_{mpc,\hat{L}_k} &= \arg \min_{U, \mathbf{X}} J(\mathbf{x}_k, U, \mathbf{X}^{\text{ref}}) |_{\hat{L}_k} \\ &= P_k \arg \min_{U, \mathbf{X}} J(\mathbf{x}_k, P_k U, \mathbf{X}^{\text{ref}}) |_{\hat{L}_k} \\ &= P_k \arg \min_{U, \mathbf{X}} J(\mathbf{x}_k, U, \mathbf{X}^{\text{ref}}) |_L \\ &= P_k \mathbf{u}_{mpc,L}.\end{aligned}\quad (22)$$

Substituting $\mathbf{u}_{mpc,\hat{L}_k}$ into the equation (20), a new formulation of the desired input $\mathbf{u}_{d,k}$ can be obtained in (23):

$$\begin{aligned}\mathbf{u}_{d,k} &= LP_k^{-1}(P_k \mathbf{u}_{mpc,L} + P_k \tilde{\mathbf{u}}_{\omega_k}) \\ &= \hat{L}_k(\mathbf{u}_{mpc,\hat{L}_k} + P_k \tilde{\mathbf{u}}_{\omega_k}).\end{aligned}\quad (23)$$

Combining the two formulations of $\mathbf{u}_{d,k}$ in (19) and (23) with the update equation of $\hat{l}_{m,k}$ in (16), we can derive the error between the estimation of fault parameters and the true value as (24a) and (24b):

$$\begin{aligned}l_m &= \frac{\hat{l}_{m,k}(\mathbf{u}_{mpc,\hat{L}_k} + P_k \tilde{\mathbf{u}}_{\omega_k})_m}{(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k} + \mathbf{e}_{\tilde{u},k})_m} \\ &= \hat{l}_{m,k} \frac{(\mathbf{u}_{mpc,\hat{L}_k})_m}{(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k})_m} \frac{(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k})_m}{(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k} + \mathbf{e}_{\tilde{u},k})_m} \\ &\quad + \frac{l_m(\tilde{\mathbf{u}}_{\omega_k})_m}{(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k} + \mathbf{e}_{\tilde{u},k})_m} \\ &= \hat{l}_{m,k+1} \frac{(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k})_m}{(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k} + \mathbf{e}_{\tilde{u},k})_m} \\ &\quad + \frac{l_m(\tilde{\mathbf{u}}_{\omega_k})_m}{(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k} + \mathbf{e}_{\tilde{u},k})_m}\end{aligned}\quad (24a)$$

$$\begin{aligned}l_m - \hat{l}_{m,k+1} &= -\hat{l}_{m,k+1} \frac{(\mathbf{e}_{\tilde{u},k})_m}{(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k} + \mathbf{e}_{\tilde{u},k})_m} \\ &\quad + \frac{l_m(\tilde{\mathbf{u}}_{\omega_k})_m}{(\mathbf{u}_{mpc,\hat{L}_k} + \mathbf{u}_{RL,k} + \mathbf{e}_{\tilde{u},k})_m} \\ &= \frac{l_m}{(\mathbf{u}_{d,k})_m} (l_m(\tilde{\mathbf{u}}_{\omega_k})_m - \hat{l}_{m,k+1}(\mathbf{e}_{\tilde{u},k})_m)\end{aligned}\quad (24b)$$

Note that $q_{m,k+1} = \frac{l_m}{\hat{l}_{m,k+1}}$, the ratio of l_m to $\hat{l}_{m,k+1}$, can be simplified based on (24b):

$$q_{m,k+1} = \frac{(\mathbf{u}_{d,k})_m - l_m(\mathbf{e}_{\tilde{u},k})_m}{(\mathbf{u}_{d,k})_m - l_m(\tilde{\mathbf{u}}_{\omega_k})_m}. \quad (25)$$

During AFTC, under the action of $\mathbf{u}_{RL,k}$, the state error will decrease with the approximation of $\hat{L}_k$ to L . With the decreasing of $\mathbf{e}_k$, the norm of $\tilde{\mathbf{u}}_{d,k}$ and $\mathbf{u}_{RL,k}$ also decrease progressively and the difference $\|\mathbf{e}_{\tilde{u},k}\|$ between them will converge to $\|\tilde{\mathbf{u}}_{\omega_k}\|$. Therefore, $q_{m,k+1}$ will converge into the interval $(1 - \epsilon, 1 + \epsilon)$ and ϵ is a minuteness related to $\|\tilde{\mathbf{u}}_{\omega_k}\|$. It means that the difference between $\hat{l}_m$ and l_m will converge into a bounded range under the iteration of (16). $\square$

Intuitively, the actuator efficiency l_m is the ratio of the energy injected into the system by the motor to the energy required by the system, so (16) has practical physical meaning. And the convergence of $\hat{l}_m$ which is proved above is the key to reducing the state error and the long-term impact of the RL controller on the system.

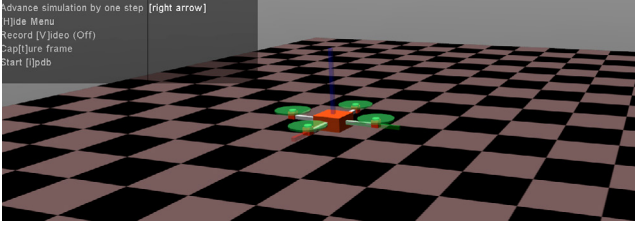


Fig. 3. Simulation environment based on Mujoco

4. SIMULATION RESULTS AND ANALYSIS

In this section, we show the effectiveness of the FDD scheme, the auxiliary controller based on RL and the results of trajectory tracking based on the proposed AFTC framework.

4.1 Simulation Environment and Physical Parameters

We build a simulation environment (shown in Fig.3) based on the Mujoco (Todorov et al. (2012)) physics engine for algorithm training and trajectory tracking simulations. In the simulation environment, we take the parameters for the quadcopter as in Table 1.

Table 1. Parameters for Quadcopter

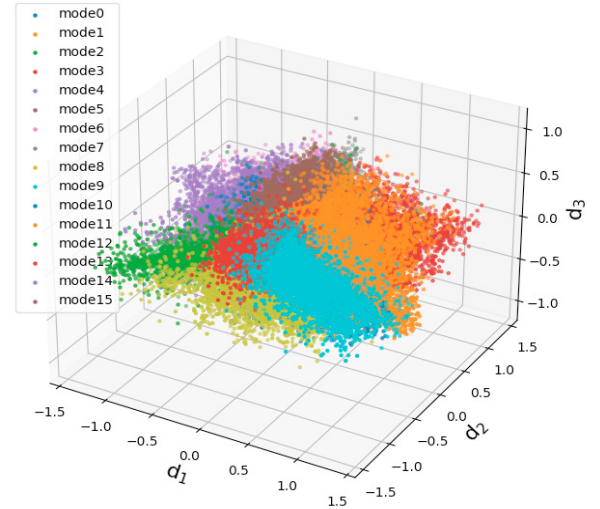
Parameters	Values
$I_{xx} = I_{yy}$	$1.216e - 3 \text{ kg.m}^2$
I_{zz}	$2.382e - 3 \text{ kg.m}^2$
m	0.28 kg
k_T	$1e - 4 \text{ N.s}^2$
b	$1e - 6 \text{ N.m.s}^2$
l	0.13 m
g	9.81 m/s^2

4.2 Solution of NMPC Optimal Control Problem

We treat the NMPC optimal control problem (OCP) in (4) as a nonlinear optimization problem and solve it with the Casadi (Andersson et al. (2019)) package. Considering the control frequency of the actual UAV and the time for solving the OCP once, we choose the prediction horizon $N_p = 10$ and a 20Hz control frequency.

4.3 Fault Detection and Diagnosis

To comprehensively analyze the distribution of state error under different fault parameters and system states, we sampled 200 sub-models ($N_q = 200$) under each fault mode and 500 system states ($N_k = 500$) in state space for each sub-model. Then a total of $16 \times 200 \times 500$ samples of state error are obtained. During sampling, the state space $\mathbb{X}$ is $\text{Box}[0.1, 0.1, 0.1, 0.5, 0.5, \pi, 5, 5, 2, 2, 2, 1]$. The distribution of samples is shown in Fig.4. Samples of different modes have specific distribution in space, but are not completely separated from each other. For fault modes with some minor faults, the samples are located in the overlapping area. We can get the discrimination model classifying the samples by SVC algorithm, and the correct rate is 91%. When there is a motor with a large fault, it is possible that the motor with a minor fault will be judged as healthy by mistake. In this case, the motor with a large fault will be compensated preferentially and it has little influence on the final fault tolerance effectiveness.

Fig. 4. The distribution of state errors under $mode_j$

4.4 Training for Auxiliary Controller Based on DDPG

We train for auxiliary controller based on DDPG algorithm. Fully connected MLPs (multilayer perception) which have two hidden layers (256×128) with linear hidden activation and tanh output activation are used as function approximators in the critic network and the actor network. It should be noted that the output dimension of the actor network is n_u^j for the agent in $mode_j$. And the learning rates of the critic network and the actor network are 0.001 and 0.0001, respectively. During sampling, we take the simulation frequency as 20Hz consistent with the main control frequency. The initial state $\mathbf{x}_0$ is sampled according to (6f) and the fault parameter $L_{mode_j, i}$ is sampled according to (6d) and (6e). Each episode has a maximum length of 100 samples and the size of replay buffer is 30,000. In order to converge faster during training, we take 64 samples as a batch for one update. The reward curves of the agent's training for different fault modes are shown in Fig.5.

From the reward curves, it can be found that the agent gradually learns to give appropriate actions to compensate for the state error, so as to obtain higher rewards. In addition, compared with $mode_{11}$, the agent for $mode_o$ requires much larger training samples, and the network convergence takes longer time due to the higher dimensions of action.

4.5 Trajectory Tracking with Actuator Faults under AFTC

In this subsection, we show the effectiveness of the proposed AFTC framework under two different fault scenarios (FS):

- 1) FS1: An abrupt loss of effectiveness occurring to motor 2 with $l_2 = 0.5$ at time $t = 6\text{sec}$ as in Fig.7.
- 2) FS2: An abrupt loss of effectiveness to all motors at time $t = 6\text{sec}$ followed by a sinusoidal fault as in Fig.10.

Considering the impact of noise in the real environment, we multiply Gaussian noise $\mathcal{N}(1, 0.005)$ on motors and

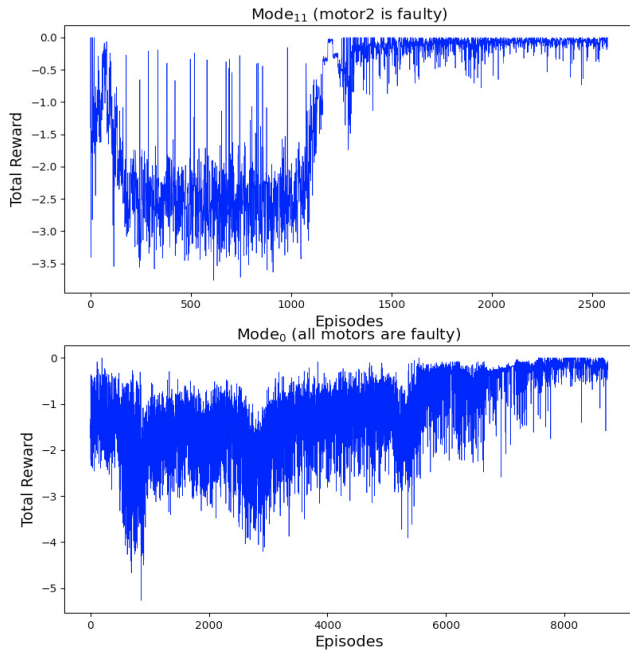


Fig. 5. The reward curves of training for $mode_0$ (only motor 2 is faulty) and $mode_{11}$ (all motors are faulty)

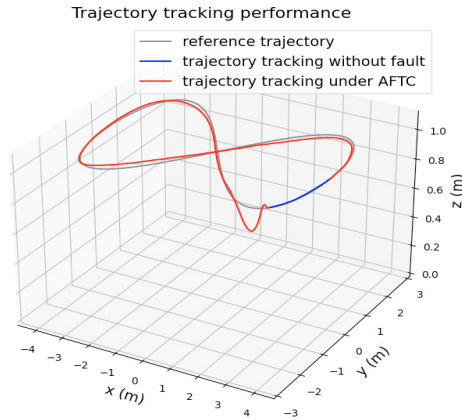


Fig. 6. The performance of trajectory tracking in fault $mode_{11}$ under AFTC

add Gaussian noise $\mathcal{N}(0, 0.01)$ to all measurement states during the simulation.

FS1: In the first fault scenario, only motor 2 experiences an abrupt fault at time $t = 6\text{sec}$ and the performance of trajectory tracking with the fault under AFTC is shown in Fig.6. When the fault occurs, the FDD module gives fault indication and information of fault mode according to the judgment of state error by the discriminant model, and the corresponding auxiliary controller is activated and gives auxiliary control signal. With the control signal, the estimation of the fault parameter is updated which is used to reconfigure the MPC controller and nominal model, and the result of the estimation is shown in Fig.7. As the estimated value converges to the real fault value, the controller gradually matches the real system resulting in the reduction of the state error. It can be seen from Fig.8 that with the recovery of the system, the auxiliary control signal decreases, thus reducing the impact of the instability of the RL controller on the system.

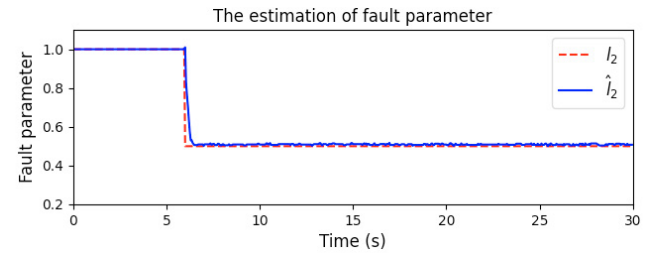


Fig. 7. The estimation of the fault parameter in $mode_{11}$

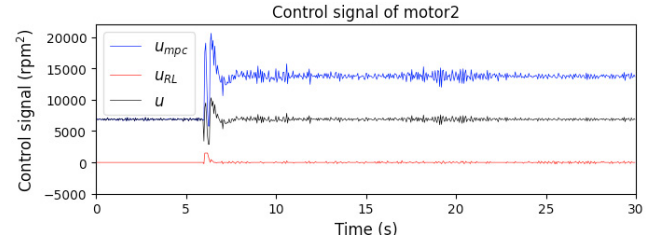


Fig. 8. Control signals for motor 2 conducted by MPC controller, RL controller and the real input of system in $mode_{11}$

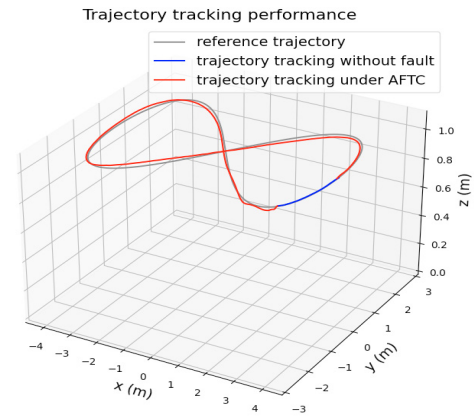


Fig. 9. The performance of trajectory tracking in fault $mode_0$ under AFTC

FS2: In the second fault scenario, all motors experience faults at time $t = 6\text{sec}$ and the fault parameter $L = \text{diag}(0.8, 0.7, 0.6, 0.5)$. At time $t = 9\text{sec}$, we add a sinusoidal fault $f_t = 0.1 - 0.1\sin(\frac{t-9}{2} + \frac{\pi}{2})$ to all motors. The AFTC process in FS2 is similar to FS1, and the performance of trajectory tracking with the faults under AFTC is shown in Fig.9. The control signals conducted by the MPC controller, RL controller and the real input of system in $mode_0$ are shown in Fig.11. Different from FS1, in the case of multiple faults, the auxiliary controller will give priority to compensating the output of the motor with major faults and suppress the output of the motor with minor faults to maintain the attitude of the quadcopter. And it will lead to an estimation great than 1 of the minor fault. Considering that $l > 1$ does not exist in the real situation, we constrain the estimated value to a reasonable range and the result of the estimation of faults is shown in Fig.10.

Based on all the simulation results above, it is verified that the proposed AFTC framework is effective enough to handle actuator faults of quadrotors and the proposed

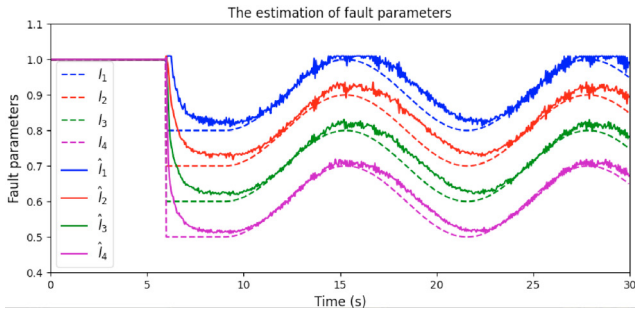


Fig. 10. The estimation of the fault parameter in $mode_0$

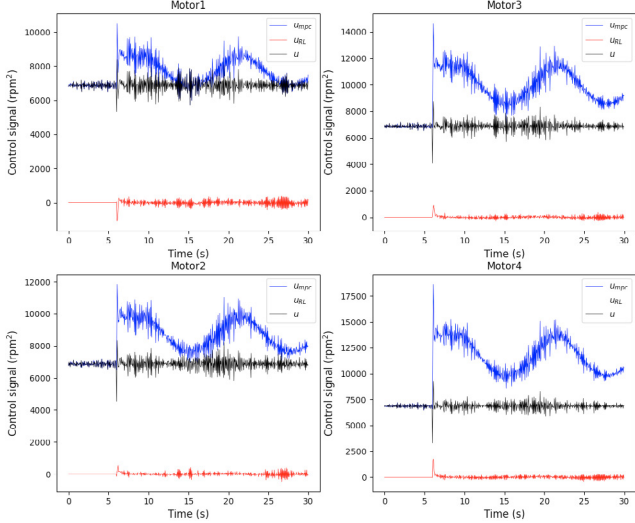


Fig. 11. Control signals conducted by MPC controller, RL controller and the real input of system in $mode_0$

fault parameter estimation method also has a satisfactory convergence performance.

5. CONCLUSIONS

In this paper, we have proposed a novel AFTC framework for the quadcopter with actuator faults. By virtue of the characteristic that the state error of the quadcopter has specific distributions under different fault modes, we use the SVC algorithm to establish the discriminant model of the state error as the FDD module. We have designed and trained a group of auxiliary controllers based on the DDPG algorithm to compensate the system input for different fault modes. As the core of this paper, we have proposed an auxiliary signal-based actuator fault parameter estimation method and proved its convergence. This estimation method can avoid solving optimization problems such as least squares and improve the real-time performance of FTC. At the same time, the proposed framework combines MPC with RL, which reduces the requirements for accurate modeling, and avoids the instability of the RL controller under continuous operation. Finally, in the case of single fault and multiple faults, trajectory tracking simulations are carried out to verify the effectiveness of the proposed active FTC.

However, due to the negative feedback structure, though the impact of instability caused by the RL controller is almost suppressed, the drawback of RL still needs to be fundamentally addressed. Additionally, more fault

situations will be considered and a real-world experiment will be conducted in the future.

REFERENCES

- Andersson, J.A., Gillis, J., Horn, G., Rawlings, J.B., and Diehl, M. (2019). Casadi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11(1), 1–36.
- Eltrabyly, A., Ichalal, D., and Mammar, S. (2021a). Fault-tolerant model predictive control trajectory tracking for a quadcopter with 4 faulty actuators. *IFAC-PapersOnLine*, 54(4), 141–146.
- Eltrabyly, A., Ichalal, D., and Mammar, S. (2021b). Quadcopter trajectory tracking in the presence of 4 faulty actuators: A nonlinear MHE and MPC approach. *IEEE Control Systems Letters*, 6, 2024–2029.
- Fei, F., Tu, Z., Xu, D., and Deng, X. (2020). Learn-to-recover: Retrofitting UAVs with reinforcement learning-assisted flight control under cyber-physical attacks. In *the 2020 IEEE International Conference on Robotics and Automation (ICRA)*, 7358–7364. Paris, France.
- Fourlas, G.K. and Karras, G.C. (2021). A survey on fault diagnosis and fault-tolerant control methods for unmanned aerial vehicles. *Machines*, 9(9), 197.
- Gunn, S.R. et al. (1998). Support vector machines for classification and regression. *ISIS technical report*, 14(1), 5–16.
- Hwangbo, J., Sa, I., Siegwart, R., and Hutter, M. (2017). Control of a quadrotor with reinforcement learning. *IEEE Robotics and Automation Letters*, 2(4), 2096–2103.
- Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. (2015). Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*.
- Luukkonen, T. (2011). Modelling and control of quadcopter. *Independent research project in applied mathematics, Espoo*, 22, 22.
- Nan, F., Sun, S., Foehn, P., and Scaramuzza, D. (2022). Nonlinear MPC for quadrotor fault-tolerant control. *IEEE Robotics and Automation Letters*, 7(2), 5047–5054.
- Rubí, B., Morcego, B., and Pérez, R. (2020). A deep reinforcement learning approach for path following on a quadrotor. In *the 18th European Control Conference (ECC)*, 1092–1098. St. Petersburg, Russia.
- Sohège, Y., Quiñones-Grueiro, M., and Provan, G. (2021). A novel hybrid approach for fault-tolerant control of UAVs based on robust reinforcement learning. In *the 2021 IEEE International Conference on Robotics and Automation (ICRA)*, 10719–10725. Xi'an, China.
- Todorov, E., Erez, T., and Tassa, Y. (2012). Mujoco: A physics engine for model-based control. In *the 2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 5026–5033. IEEE.
- Yu, B., Zhang, Y., and Qu, Y. (2015). MPC-based FTC with FDD against actuator faults of UAVs. In *the 15th International Conference on Control, Automation and Systems (ICCAS)*, 225–230. Busan, South Korea.